

参与项目情况报告

1.1 项目基本情况

1.1.1 项目概述

项目名称：编程平台后端系统 (program_platform_backend)

项目定位：在线编程评测平台的后端服务，提供用户认证、题目管理、代码评测等核心功能

技术栈：Spring Boot 2.7.18 + MyBatis + MySQL 8.0 + JWT + Knife4j + 阿里云 OSS + 火山大模型 (豆包 Seed)

1.1.2 项目架构

架构风格：集成式单体应用 (Integrated Monolith)

模块划分：

用户认证模块 (Auth)：负责用户登录、注册、Token 管理

用户中心模块 (User)：用户信息管理、学习统计、提交记录

题目管理模块 (Problem)：题目 CRUD、测试用例管理

评测模块 (Judge)：代码编译、运行、结果判定

算法模块 (Algorithm)：AI 辅助代码分析与题目生成

1.1.3 核心技术特性

安全认证：基于 JWT 的无状态身份认证机制

文件存储：阿里云 OSS 实现头像等文件的云端存储

代码评测：Docker 容器化沙箱环境，支持 C++、Java、Python 多语言评测

AI 能力：集成火山大模型，支持代码分析和题目生成

1.2 承担的主要工作情况

1.2.1 前期项目架构设计

作为核心开发人员之一，参与了项目的整体架构设计工作：

工作内容	职责描述	完成情况
技术方案选型	主导后端技术栈选型 (Spring Boot + MyBatis)，设计数据库 表结构	完成
架构文档编写	编写《技术架构设计文 档》，定义模块划分、接 口规范、数据流向	完成
安全架构设计	设计 JWT 认证机制、 Token 拦截器、用户上 下文管理方案	完成
数据库设计	设计用户表、提交记录 表、题目表等核心数据 表结构	完成

1.2.2 后端接口开发

(1) 用户认证模块 (AuthController)

接口名称	HTTP 方法	接口路径	功能描述
用户登录	POST	/api/v1/auth/login	验证用户凭证，生成 JWT Token
用户注册	POST	/api/v1/auth/register	创建新用户，密码加密存储
退出登录	POST	/api/v1/auth/logout	销毁当前 Token，清理用户上下文

(2) 用户个人中心模块 (UserController)

接口名称	HTTP 方法	接口路径	功能描述
获取用户信息	GET	/api/v1/user/info	查询当前登录用户的基本信息
修改用户信息	PUT	/api/v1/user/info	更新用户昵称、邮箱等信息
上传头像	POST	/api/v1/user/avatar/upload	上传头像至阿里云 OSS
修改密码	PUT	/api/v1/user/password	验证旧密码后更新新密码
获取提交记录列表	GET	/api/v1/user/submission/list	分页查询用户代码提交历史
获取提交记录详情	GET	/api/v1/user/submission/list	分页查询用户代码提交历史
获取学习统计	GET	/api/v1/user/study/statistics	获取用户学习进度统计数据

1.2.3 后期全局协助

参与代码 Review，协助团队成员排查接口问题

协助解决跨模块集成问题（如认证与评测模块的协作）

参与测试用例编写和接口联调工作

协助文档编写和技术方案优化

1.3 项目实施过程中遇到的问题及处理结果

1.3.1 问题一：Token 拦截器与跨域配置冲突

问题描述：JWT Token 拦截器与 CORS 配置存在冲突，导致前端无法正常获取 Token

分析原因：拦截器在 CORS 预检请求（OPTIONS）阶段就进行 Token 验证，导致预检请求失败

解决方案：在 TokenInterceptor 中添加 OPTIONS 请求的放行逻辑，允许预检请求直接通过

处理结果：跨域问题解决，前端正常获取和携带 Token

1.3.2 问题二：阿里云 OSS 文件上传失败

问题描述：头像上传接口偶尔出现上传失败，返回超时错误

分析原因：OSS SDK 默认超时时间较短，网络波动时容易超时

解决方案：配置 OSS 客户端的连接超时和读取超时参数，增加重试机制

处理结果：上传成功率提升至 99.9%以上

1.3.3 问题三：用户上下文获取异常

问题描述：在异步线程中无法获取当前登录用户信息

分析原因：UserContext 基于 ThreadLocal 实现，异步线程无法继承父线程的 ThreadLocal 变量

解决方案：将 UserContext 改为基于请求头 Token 的动态获取方式

处理结果：异步场景下用户信息获取正常

1.4 参与项目过程的体会与项目评价

1.4.1 项目体会

(1) 架构设计的重要性

前期的架构设计为项目奠定了良好的基础，清晰的模块划分和接口规范使得后续开发更加高效。

(2) 安全认证的复杂性

JWT 认证机制虽然实现了无状态认证，但在 Token 刷新、过期处理、黑名单管理等方面仍有优化空间。

(3) 团队协作的关键

项目的顺利推进离不开团队成员的密切配合，良好的接口文档和及时的联调反馈是项目成功的保障。

1.4.2 项目评价

优点：

技术选型合理，采用成熟稳定的 Spring Boot 生态

架构设计清晰，模块职责明确

代码规范统一，注释完善，便于维护

集成了 Swagger/Knife4j，接口文档自动化

改进建议：

引入 Redis 缓存，优化高频查询接口性能

完善日志体系，增加链路追踪能力

补充单元测试和集成测试，提升代码覆盖率

1.4.3 个人收获

通过全程参与编程平台后端系统的开发，我在技术能力、项目经验和团队协作等方面都获得了显著提升，具体体现在以下几个方面：

(1) 架构设计能力的深度锻炼

作为项目前期架构设计的核心成员，我系统地学习了如何从 0 到 1 构建一个中型后端系统。从技术选型到模块划分，从数据库设计到接口规范制定，整个过程让我对“高内聚、低耦合”的架构原则有了更深刻的理解。特别是在设计用户认证模块时，需要综合考虑安全性、可扩展性和性能等多方面因素，这让我学会了在多种方案中进行权衡取舍。

(2) Spring Boot 生态的全面实践

在项目开发过程中，我深入实践了 Spring Boot 框架的核心技术栈。从依赖注入、AOP 切面编程到拦截器的自定义实现，从 MyBatis 的 CRUD 操作到复杂 SQL 的优化，从 JWT Token 的生成验证到用户上下文的 ThreadLocal 管理，每一个环节都让我对 Spring Boot 生态有了更扎实的掌握。同时，通过集成 Knife4j、阿里云 OSS 等第三方组件，我也学会了如何将外部服务与 Spring Boot 应用无缝对接。

(3) 安全认证机制的深入理解

在实现用户认证模块时，我系统地学习了 JWT 认证的原理和最佳实践。从 Token 的生成、解析到过期处理，从密码的 BCrypt 加密存储到登录状态的管理，每一个细节都涉及到系统的安全性。特别是在处理 Token 拦截器与 CORS 配置冲突的问题时，让我对 HTTP 请求的处理流程有了更清晰的认识。

(4) 问题排查与解决能力的提升

项目开发过程中遇到的各种技术问题，如阿里云 OSS 文件上传超时、异步线程中用户上下文获取异常等，都锻炼了我的问题排查能力。通过查看日志、调试代码、查阅官方文档等方式，我学会了如何快速定位问题根源并提出有效的解决方案。这种能力不仅适用于当前项目，更是未来开发工作中的宝贵财富。

(5) 团队协作与沟通能力的增强

作为团队的核心成员之一，我深刻体会到了团队协作的重要性。在与前端开发人员对接接口时，我学会了如何编写清晰的接口文档；在代码 Review 过程中，我学会了如何提出建设性的改进意见；在跨模块集成测试时，我学会了与其他模块开发人员密切配合。这些经历让我明白，一个优秀的开发人员不仅需要具备扎实的技术能力，还需要具备良好的沟通能力和团队协作精神。